

Selection and Randomized Quickselect

Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**

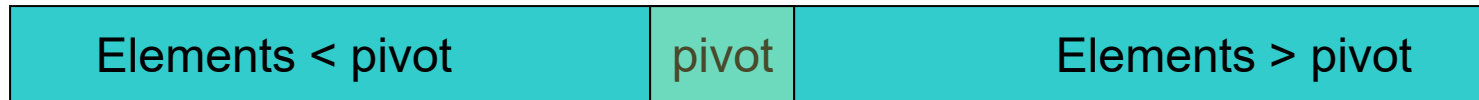


Selection

- **Selection** is the problem of locating the k th largest element in an unsorted array / linked list.
- The k th largest element is also called the k th **order statistic**.
- Common values of k :
 - $k = 1$: minimum
 - $k = n$: maximum
 - $k = n/2$: median
- It's easy to find the min and max in $\Theta(n)$ time.
- For the median, we can easily achieve $\Theta(n \log n)$ time by first sorting the array, but can we do it faster?

“Quickselect”

- To select for the item of **rank** k in an array $A[1..n]$.
- As in quicksort, pick a pivot element and partition A in linear time:



- After partitioning, the pivot element ends up being placed where it would in the sorted ordering of A
 - So we know the rank, r , of the pivot!
 - If $k = r$, the pivot is the element we seek and we're done.
 - If $k < r$, select for the element of rank k on the left side.
 - If $k > r$, select for the element of rank $k - r$ on the right side.
- **Just like quicksort, except we only recurse on one subproblem instead of both.**

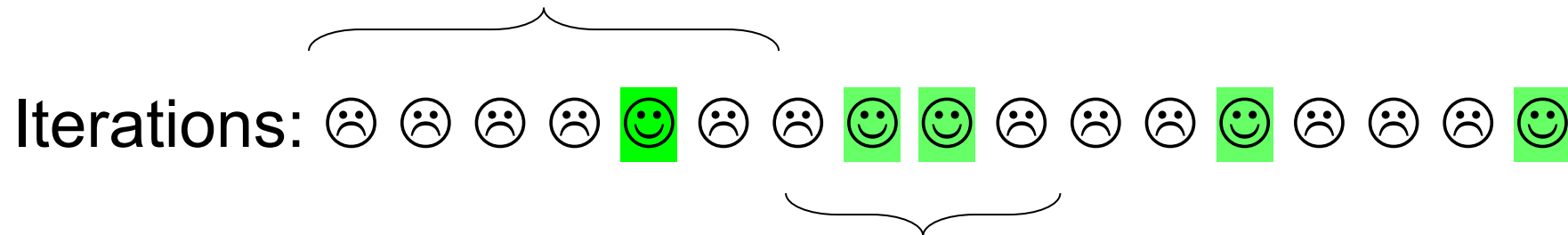
Randomized Quickselect

- As in quicksort, the challenge with quickselect is choosing a good pivot.
- Ideally use the median, but that's a chicken and egg problem, since we're trying to compute the median!
- Good choice: choose a random element as the pivot. This gives an expected running time of $\Theta(n)$ (worse case is still $\Theta(n^2)$).

Randomized Quickselect

- As in quicksort, the challenge with quickselect is choosing a good pivot.
- Good choice: choose a random element as the pivot. This gives an expected running time of $\Theta(n)$ (worse case is still $\Theta(n^2)$).

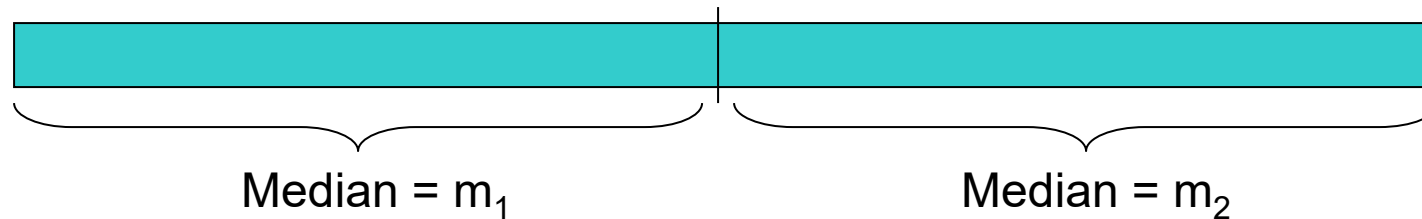
$\Pr[\text{reduction to } \leq 2/3 \text{ size}] \geq 1/3 \rightarrow$ at most 3 expected iterations, each taking $\leq n$ units of time



- Expected total iterations: $\leq 3(n + (2/3)n + (2/3)^2n + \dots) = \Theta(n)$

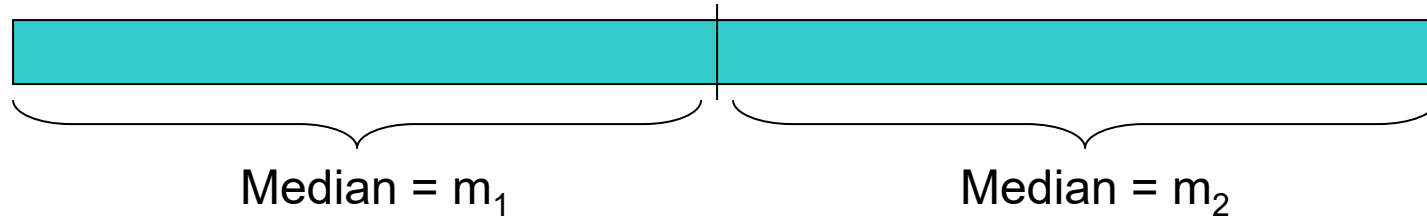
Deterministic Selection

- Is there a $\Theta(n)$ deterministic selection algorithm?
 - This fundamental problem remained open for some time before finally being resolved (positively) in the early 1970s.
- How might we solve this using divide and conquer?
 - Focus on finding the median for now.
 - Take an array $A[1..n]$, recursively find median of its first and second halves:

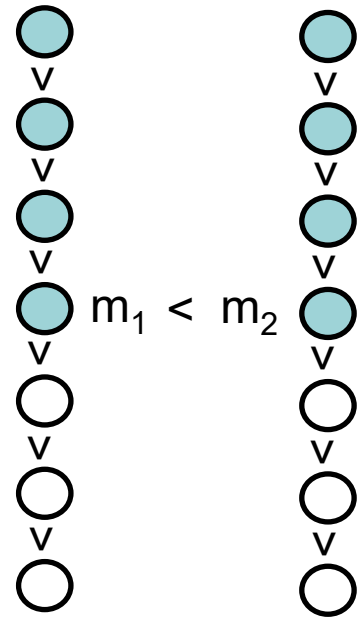


- Where is the median of the entire array A compared to m_1 and m_2 ?

Deterministic Selection



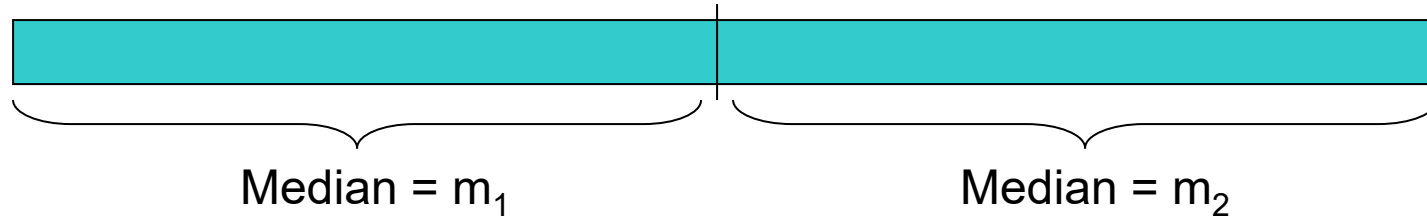
- Claim: The median of A lies between $\min(m_1, m_2)$ and $\max(m_1, m_2)$.



How many elements are $\geq m_1$?

More than half!

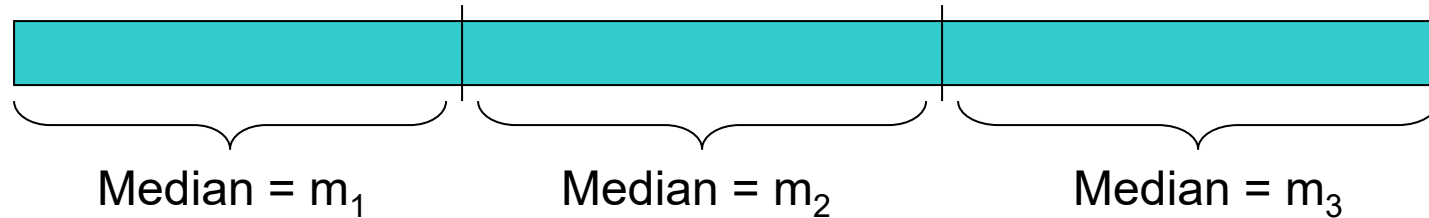
Deterministic Selection



- Claim: The median of A lies between $\min(m_1, m_2)$ and $\max(m_1, m_2)$.
- For block with the smaller median, discard elements $<$ the median.
- For block with the larger median, discard elements $>$ the median.
- We discard $n/4$ elements $<$ $\text{median}(A)$ and $n/4$ elements $>$ $\text{median}(A)$, so this doesn't change the median of A !
- Now recursively select for the median of the remaining $n/2$ elements.
- What is the total running time? $T(n) = 3T(n/2) + \Theta(n) \rightarrow \underline{O(n^{1.58})}$ ☹️

Deterministic Selection, Take 2...

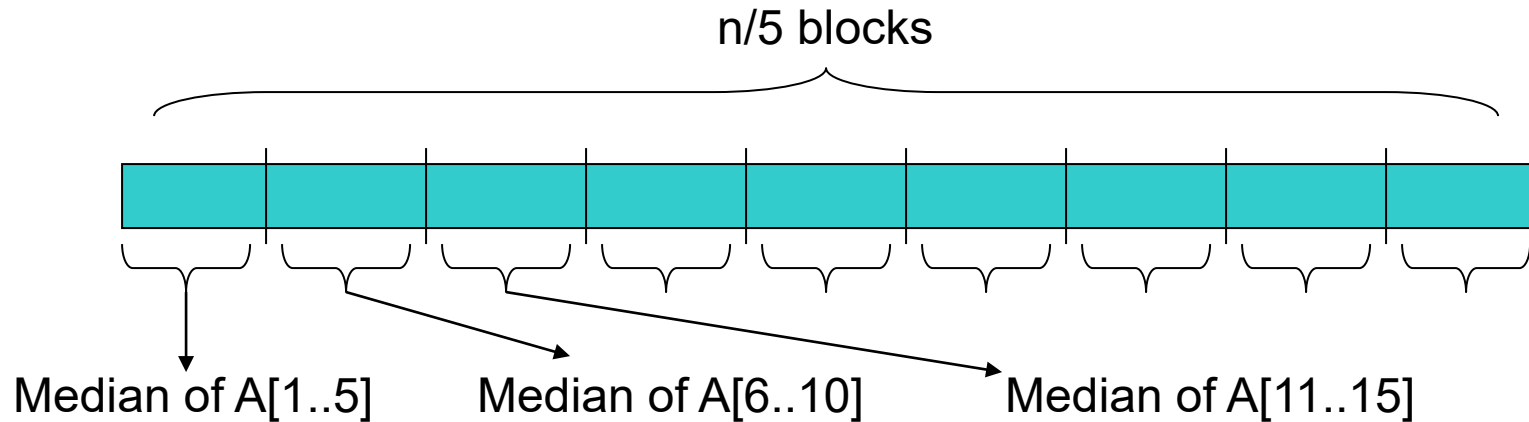
- Ok, so the obvious “divide in half” approach didn’t give a fast enough running time. What if we divide into 3 pieces?



- Sadly, this doesn’t work either...

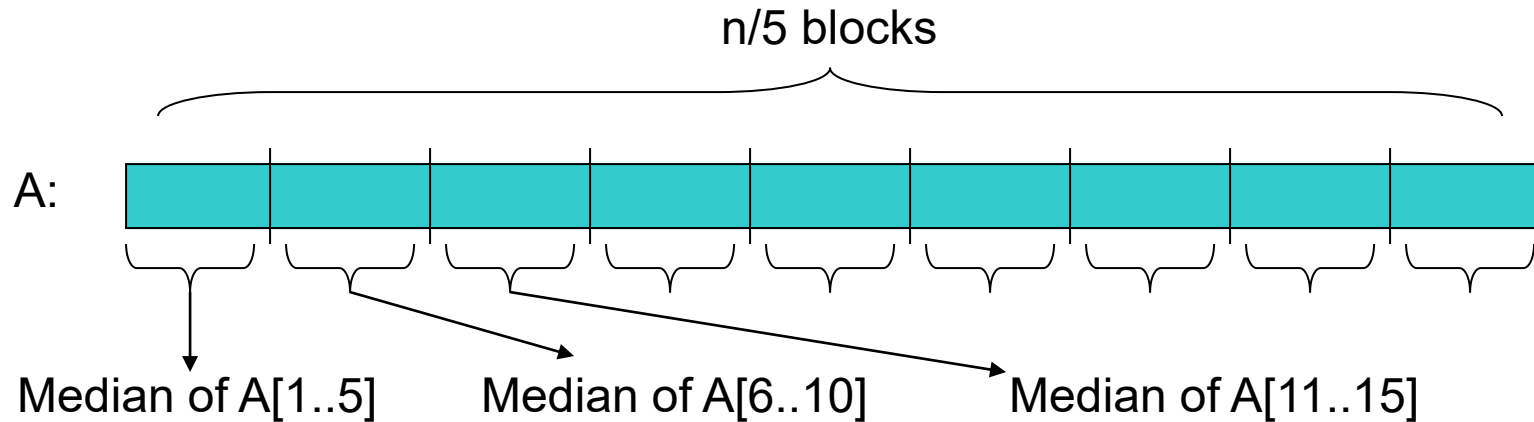
Deterministic Selection, Take 3...

- But remarkably, this works:



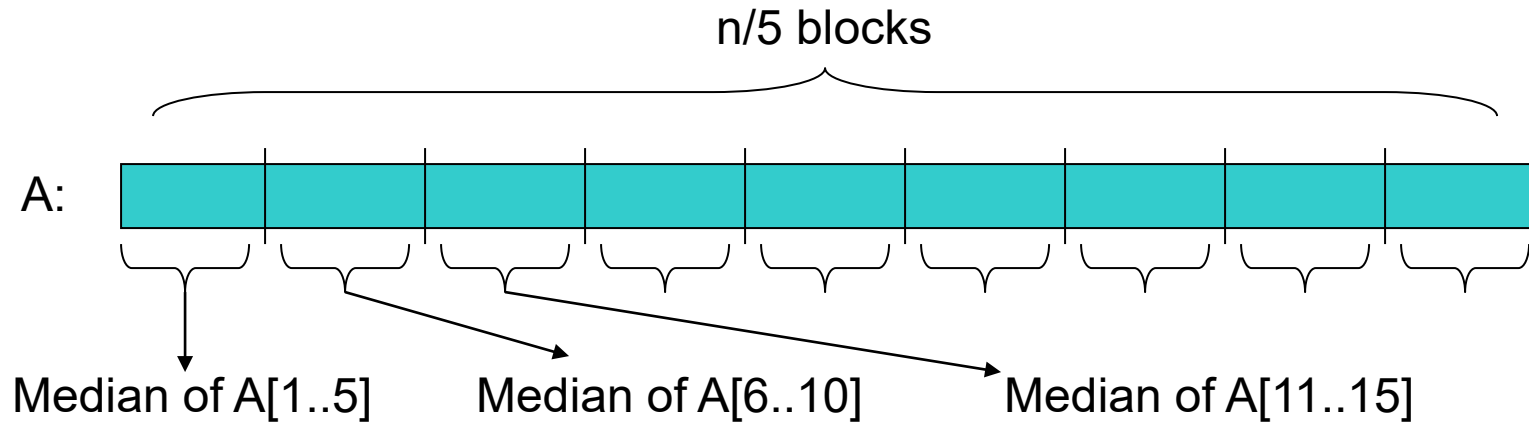
- Divide A into $n/5$ blocks of length 5.
- Find median of each block (only $\Theta(n)$ total time!).
- Recursively find the median M of the block medians ($T(n/5)$ time).
- This is a good enough choice for the pivot for quickselect!

Deterministic Selection

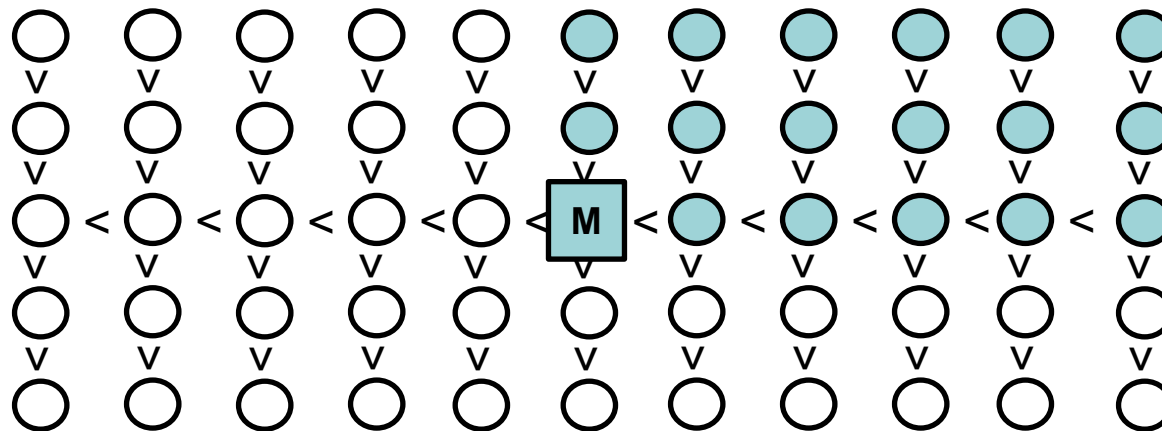


- M = median of block medians.
- **Claim:** At least $3/10$ of the elements of A are $\leq M$, and at least $3/10$ of the elements of A are $\geq M$.
- So if we use M as a pivot in quickselect, we will recurse on a subproblem of size at most $7n/10$.
- Total runtime: $T(n) \leq T(n/5) + T(7n/10) + \Theta(n)$.
 - Decreasing series! Solves to $T(n) = \Theta(n)$.

Deterministic Selection



- M = median of block medians.
- Claim: at least $3/10$ of the elements of A are $\leq M$, and at least $3/10$ of the elements of A are $\geq M$.



Deterministic Median Finding: Applications

- The median is an ideal partitioning point for divide and conquer algorithms.
- **Example:** $\Theta(n)$ deterministic version of quickselect for finding any order statistic. $T(n) = T(n/2) + \Theta(n)$.
- **Example:** $\Theta(n \log n)$ deterministic version of quicksort.
(Although, somewhat high hidden constant, so not very competitive)